LAB 7: Fuzzing for Input Validation Bugs

Car Rental Rate — Lab 7 submission

Student SRN: PES2UG24CS003

1. Objective

Apply property-based fuzz testing with Hypothesis to discover edge-case failures in three input-processing functions, then harden the application and verify expected behaviour.

2. Functions under test

Function	Purpose
sanitize_license_plate	Remove supported separators and uppercase a license plate.
parse_trip_mileage	Parse comma-separated mileage values into floats.
format_rental_invoice	Parse Driver:Days:Category and calculate the rental total.

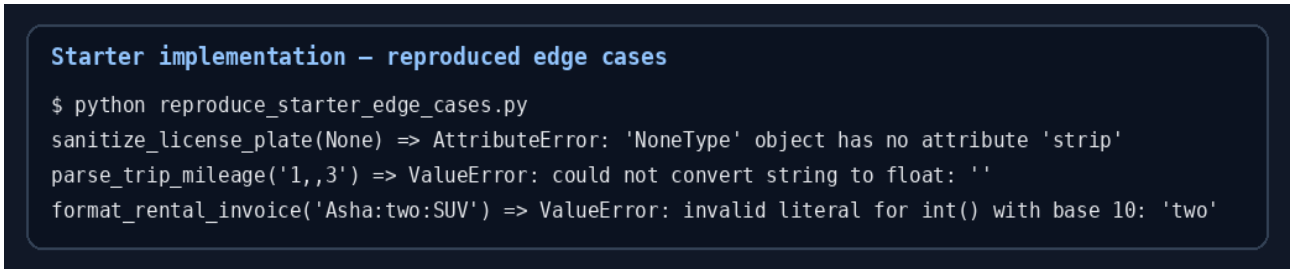
3. Bugs reproduced in the starter code

The supplied starter assumes every argument is a correctly formatted string. The following invalid inputs reproduce failures in the original functions:

```
$ python reproduce_starter_edge_cases.py
sanitize_license_plate(None) => AttributeError: 'NoneType' object has no attribute 'strip'
parse_trip_mileage('1,,3') => ValueError: could not convert string to float: ''
format_rental_invoice('Asha:two:SUV') => ValueError: invalid literal for int() with base 10: 'two'
```

The observed failures include calling .strip() on None, converting an empty mileage token to float, and converting non-numeric day text to int.

Starter edge-case evidence



4. Fixes applied

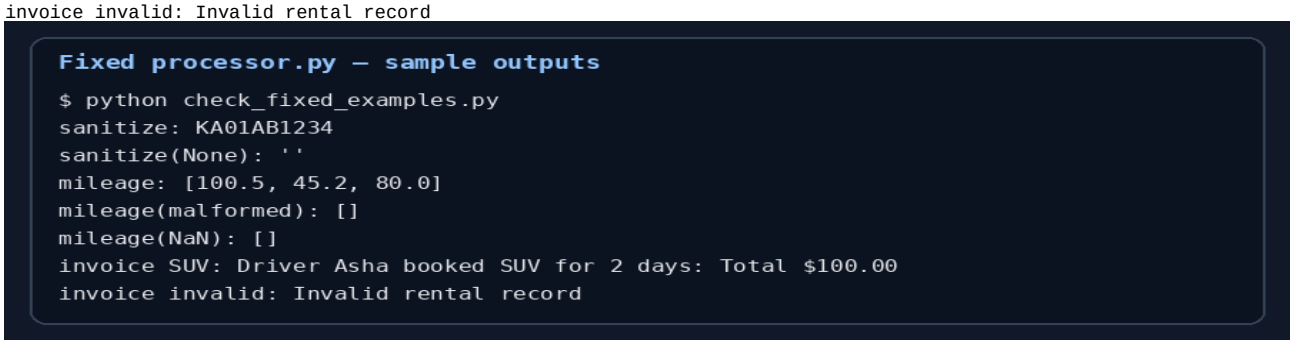
License plate: non-string input returns an empty string. The original delimiter removal and uppercase behaviour is preserved.

Mileage parser: validates input type, handles empty input, rejects empty tokens, malformed numbers, negative values, NaN and infinity. Invalid input returns an empty list rather than raising an exception or returning partial data.

Invoice formatter: validates type and exact field count; requires non-empty driver/category and a positive integer day count. Invalid records return “Invalid rental record”. Original rates are preserved: SUV $50/day; all other categories $30/day.

5. Fixed application sample output

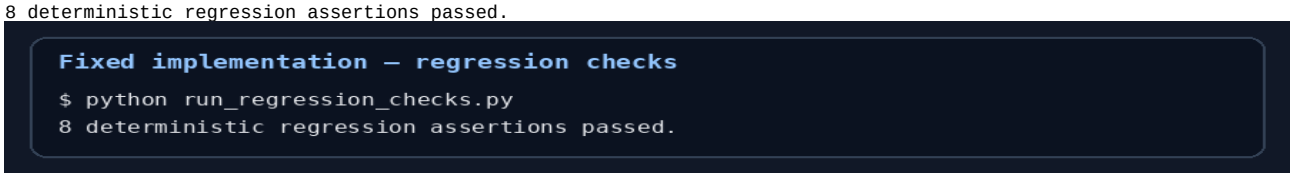
```
$ python check_fixed_examples.py
sanitize: KA01AB1234
sanitize(None): ''
mileage: [100.5, 45.2, 80.0]
mileage(malformed): []
mileage(NaN): []
invoice SUV: Driver Asha booked SUV for 2 days: Total $100.00
invoice invalid: Invalid rental record
```



6. Regression checks

Eight deterministic regression assertions were executed against the fixed implementation, covering valid and invalid values across all three functions.

```
$ python run_regression_checks.py
8 deterministic regression assertions passed.
```



7. Hypothesis property-based tests

The submitted test_processor.py uses Hypothesis to generate arbitrary text and None values for all three functions. It checks no-crash behaviour and output invariants. An additional generated valid-record property checks invoice calculations; explicit regression tests cover malformed CSV values, non-finite mileage, invalid day counts and missing fields.

Important execution note: Hypothesis was not installed in the report-generation environment. The property-based tests are included but have not been represented as a completed Hypothesis pytest run. Install the dependencies with `pip install -r requirements.txt`, then run `python -m pytest -v test_processor.py` on Ubuntu to execute them and capture the actual pytest result.

8. Conclusion

The fixed functions now handle invalid types, malformed fields and important numeric edge cases without uncaught exceptions. The original rental pricing rule is retained. The included Hypothesis suite provides generated-input coverage once run in the configured Python environment.